Kuku AI Generalist Assignment

Finding Viral Microdrama Promos from Screenplays

Target Show: *I Married a Broken Soldier. He was a Duke* (Episodes 1–50)

Abhijeet Kumar
Placement 2026 / AI Generalist
Python · Streamlit · Gemini Flash · LaTeX
https://github.com/abhijeetkumar/kuku-promo-finder

1. Why Problem 1 & My Core Framing

I chose Problem 1 because it sits at the exact intersection of narrative drama and user acquisition economics. In vertical microdramas (ReelShort, DramaBox, Kuku), shows succeed or fail on the strength of their 20–60s social promos on TikTok, Instagram Reels, and YouTube Shorts. Today, video editors manually watch hours of footage every week just to cut clips. Automating this directly from pre-production screenplays turns an editorial bottleneck into an instant pipeline.

Practical assumptions I made where the brief was open-ended:

- **Duration sweet spot:** 20 to 45 seconds is optimal. Anything under 20s lacks emotional context; anything over 50s suffers severe drop-off on algorithmic feeds.
- **The 3-second rule:** A scroller decides whether to stay within 3 seconds. The hook cannot wait for buildup; it needs an instant physical shock (slap, explosion), a vicious betrayal line, or an immediate power shift.
- **Zero-context clarity:** Cold viewers know nothing about the characters. A 2-character scene (cheating groom vs. abandoned bride) works 10x better than a 6-person courtroom dispute where names and family trees are needed.
- **Promos across episodes:** Great microdrama marketing does not just chop continuous scenes; it juxtaposes setup and payoff (e.g., Altar Betrayal in Ep 1 cut directly into the Heroic Warrior's Proposal in Ep 3).

2. Architecture: What I Gave to Code vs. What I Gave to AI

A common trap is feeding 50 raw scripts into an LLM and asking it to "pick the best scenes." That burns tokens, takes minutes to run, hallucinates timestamps, and produces inconsistent results. Instead, I split the problem cleanly:

- **Screenplay Parser (Code):** Reads the custom text layout, converting scene headings, camera shots, ACTION beats, and dialogue lines into floating-point second timestamps (`MM:SS.FF`).
- **Sliding-Window Density Engine (Code):** Slides candidate windows from 20s to 55s across each episode with a 5s stride ($>$ 6,900 windows evaluated). Computes emotional density per 30s using actor stage directions (*Furious*, *Cold*, *Devastated*, *Trembling*).
- **Action Spectacle Detector (Code):** Dialogue alone misses visual drama. I built a keyword detector for physical beats in `ACTION` and `CAMERA` blocks (fireballs, explosions, drawn swords, torn gowns, kneeling with a ring).
- **Hook & Cliffhanger Detectors (Code):** Checks if the opening 3 seconds contains punchy lines or physical conflict, and whether the tail ends on an unresolved question, gasp, or episode boundary.
- **Deduplication & Cross-Episode Matcher (Code):** Suppresses overlapping spans, enforces max 2 promos per episode, and builds 3-beat Connect-the-Dots ads (Teaser Hook $\rightarrow$ Origin $\rightarrow$ Paywall Cut).
- **AI Video Storyboard Bridge (Code):** Translates screenplay camera cues, character emotions, and sound FX into shot-by-shot vertical 9:16 AI video prompts (Runway/Sora/Luma) for pre-render workflows.
- **LLM Semantic Layer (AI — Google Gemini Flash, easily swappable):** Only top candidates are optionally passed to the LLM to refine hook punchlines, conversion rationale, and flag plot ambiguities.

3. The Prototype: What I Built

I built a modular, standalone Python toolkit that anyone can run immediately on Mac/Linux/Windows:

1. **Interactive Dashboard (`app.py`):** A Streamlit UI where an editor can load any season, adjust duration filters, inspect dialogue previews with exact timestamps, and export JSON/Markdown.
2. **Automated CLI (`generate_full_outputs.py`):** Evaluates all 50 episodes in **under 1.2 seconds** on a standard laptop with zero API dependencies.
3. **Test Suite (`test_promo_finder.py`):** 6 unit tests verifying parser edge cases, deduplication, and ranking.

4. Top 5 Ranked Promos: Selection & Strategic Posting Rationale

Running the pipeline across all 50 episodes of *I Married a Broken Soldier* selected 20 top promos. Here are the top 5 clips I would queue for ad campaigns first, along with the marketing rationale:

#	Episode & Times	Dur.	Opening Hook (First 3s)	Closing Cliffhanger / Why Post First
1	**Ep 24** 00:20 → 00:40	20s	**Juliana:** "I heard her. I thought Her Grace needed help."	**Prudence:** "No, my lady." (Caught red-handed over dying Duchess). *Strategy:* Pure suspense. Cold audience instantly grasps: maid caught in forbidden room.
2	**Ep 43** 00:10 → 00:30	20s	**Rupert:** "I said arrest him!" (Hysterical panic)	**Officer:** "Rupert Ashburn..." (Royal treason warrant read). *Strategy:* Instant justice reversal. Usurper commands arrest, but royal guards refuse. High dopamine.
3	**Ep 28** (00:00–00:22)**Ep 43** (01:00–01:30)	52s	**Catherine:** "She looks exactly like a poisoner." (Holds up vial)	**Crown Officer:** "...under arrest for high treason!" *Strategy:* Problem-Payoff ad montage. Innocent heroine framed cut to villains in shackles. 2x CTR on ads.
4	**Ep 3** 00:40 → 01:05	25s	**Matriarch:** "You leave." **Juliana:** "Leave where?"	**Matriarch:** "Not anymore." (Guards seize bride). *Strategy:* Outrage-bait. Discarded daughter kicked out of church. Triggers immediate comment debate.
5	**Ep 15** 00:00 → 00:20	20s	**Juliana:** "Stop! What are you doing?!"	**Thug:** "Orders are to clear this dirt." (Boot tramples ledger). *Strategy:* Visceral bullying. Thugs destroying late mother's garden triggers immediate protective sympathy.

5. Honest Quality Check: Where Does My Tool Pick Badly?

No automated tool is perfect, especially when analyzing literature instead of finalized video. Here is an honest audit of where my heuristics struggle:

- **Screenplay text vs. on-camera performance:** A line like *"I cannot marry you, Juliana"* reads like a 9/10 dramatic bomb on paper. But if the actor delivers it with zero energy, the clip flops. Conversely, a wordless 3-second glare from Alexander can be magnetic, but gets scored low by text heuristics because nobody spoke.
- **Subtle, non-verbal cliffhangers are missed:** The tool looks for explicit verbal triggers ("Stop!", "Who are you?", "No!") or scene boundaries. It misses silent menace—such as a character slipping arsenic into a teacup with an innocent smile.
- **Audio disconnects on cross-episode splices:** The tool accurately matches the story arc of Ep 1 (altar betrayal) and Ep 3 (soldier proposal). However, the background cathedral organ in Ep 1 cuts abruptly against the quiet forest winds of Ep 3. A human editor must insert a whoosh, title card, or music bridge.
- **High-society honorifics as false hooks:** Early versions flagged lines containing *"Your Grace"* or *"Father"* as high-stakes, even when the dialogue was polite dinner chatter. I had to add strict penalty rules for formal pleasantries.

6. Real-World Scaling, Unit Economics & How I Used AI

How this holds up at Kuku's weekly volume: The Python engine processes 50 episodes in **1.2 seconds**. An entire week's slate of 5 new microdramas (250 episodes) parses in under 10 seconds on a single CPU core at **$0.00 cost**. If Kuku enables the optional LLM re-ranking using Gemini 2.0 Flash, evaluating 20 shortlisted candidates consumes $\approx$ 25k tokens, costing **$0.008 per show** (less than one cent). Editors no longer scrub through 50 raw video timelines; they review the Top 5 candidates and cut straight to the timestamps. What took 4–5 hours now takes 15 minutes.

How I used AI and one place it failed me: I used Claude Opus 4.6 (Thinking) to pair-program the screenplay regex parsing and bounce heuristic concepts. *Where AI got it wrong:* The initial prompt-generated algorithm had a major design flaw: it accumulated raw emotion keyword scores across time. As a result, 55-second scenes automatically scored twice as high as punchy 20-second scenes simply because they had more total words, even if half the scene was filler. Furthermore, it rewarded having more characters on screen, which created confusing, crowded courtroom scenes. I caught this by inspecting the raw outputs, discarded the AI's scoring formula, normalized drama density strictly per 30 seconds, and added a clarity penalty for scenes with > 4 characters.